

SESSION - 16

Coding with Eclipse/IntelliJ



Table of Contents

Table of Contents **1**

Coding with Ecplise/IntelliJ ***2***

Exercise 1: Import project into eclipse 2
Exercise 2: Build the mysite project and install in AEM 4
Exercise 3: Import code developed into CRXDE to my site project 4
Exercise 4: Write code into Eclipse and refer in your component html 5
Exercise 5: Write business logic into your sling model and access it on page 6
Exercise 6: Various Type of Java classes which will be used into actual projects 6

Coding with Ecplise/IntelliJ

Introduction

Development becomes easy if we work with the right Integrated Development environment tool. Here in this session we will see how to use Either Eclipse for the development (Once you are familiar with using Eclipse you can yourself learn IntelliJ).

In previous session we have completed:

- installed Eclipse in our system.
- installed AEM plugin as well as AEM developer tool.
- Created an AEM project using archetype 23.

Objectives

After completing this module, you will be able to:

- Import project into Eclipse
- Import and export code from CRXDE to Eclipse or vice versa.
- Write Java backend code into Eclipse
- Generate code bundle from Eclipse
- Install bundle into AEM server
- Understand Sling Models
- High level explanation of
    i) Sling Servlet
    ii) Workflows
    iii) Schedulars
    iv) Sling Models
    v) Junit
- How actual project implementation works

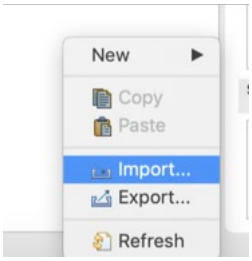
Note: We will learn all above points using exercises.

Exercise 1: Import project into eclipse

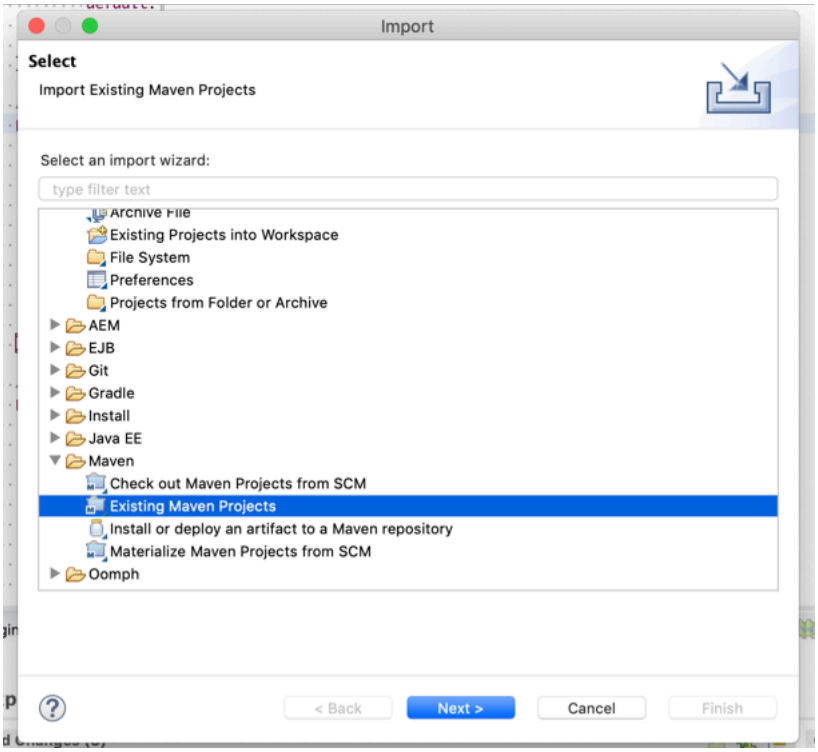
We will be working on projects created using maven. Hence, we will learn how to import a maven project into eclipse and then will work on that project.

To import a maven project follow below steps:

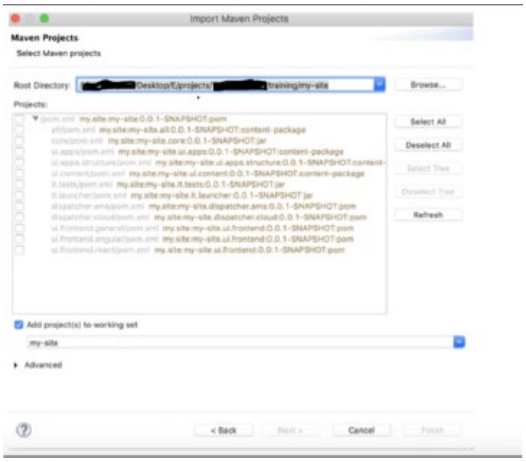
1. First copy the mysite project from exercise folder.
2. Right click in navigator window and click on import



3. Select **existing maven project** as shown:



4. Select your project root directory as shown:



5. Click on next and finish.

Your project is imported successfully.

Exercise 2: Build the mysite project and install in AEM

We will use maven commands to build and deploy this project into AEM

To build and deploy mysite project into you running server follow below steps:

1. Navigate to the root directory of mysite project and open command prompt/terminal
2. Run this maven command

   mvn clean install -PautoInstallPackage
3. Above command my site code(package and bundle) into AEM author instance.
4. if code also has to be deployed onto publish instance as well then add this profile into the command

   -PautoInstallPackagePublish

   Example -: **mvn clean install -PautoInstallPackage -PautoInstallPackagePublish**

Note: above command will run properly only when u will have **settings.xml** file with proper data in your **.m2** folder.

Once your code from above command ran successfully see the bundle and package is deployed into AEM server.

Exercise 3: Import code developed into CRXDE to my site project

This can be achieved using two ways

1. Use packages to import CRXDE code into eclipse/file system.
2. Use AEM dev tool to import CRXDE code into eclipse/files system.

We will see first approach. Second approach will be discussed later once we understand this concept.

To do this

1. we will have to create packages with proper filter.
2. Once package is created download the package and unzip the package.
3. Now copy the folder structure from the package to the file system.
4. Once code is copied into file system It should be available into eclipse as well.

Sling Models

Sling Models are annotation driven Java "POJO's" (Plain Old Java Objects) that facilitate the mapping of data from the JCR to Java variables, and provide a number of other niceties when developing in the context of AEM.
Sling models are the helper classes for frontend HTL files. Business logic is written into sling models and the output of the business class is provided to HTL via getter methods of sling models.

Reference : https://sling.apache.org/documentation/bundles/models.html

Sling Model Exporter

Sling Model Exporter was introduced in Sling Models v1.3.0. This new feature allows new annotations to be added to Sling Models that define how the Model an can be exported as a different Java object, or more commonly, serialized into a different format such as JSON.

Apache Sling provides a Jackson JSON exporter to cover the most common case of exporting Sling Models as JSON objects for consumption by programmatic web consumers such as other web services and JavaScript applications.

Reference for Sling model exporter -: https://experienceleague.adobe.com/docs/experience-manager-learn/foundation/development/develop-sling-model-exporter.html?lang=en

Exercise 4: Write code into Eclipse and refer in your component html

For this exercise we will use the earlier created components and the existing sling model which is provided into mysite project.

1. Create a new file with name in title.html in title component.
2. In that file use the HelloWorldModel.java class. Using data-sly-use attribute
3.
```
<div data-sly-use.title="my.site.my_site.core.models.HelloWorldModel">
<h1>Here is the message from sling model -: ${title.message}</h1>
</div>
```
4. Access the variable from the sling model into html.
5. Now deploy your code from mysite project.
6. Now drag and drop the title component on the page and see the output on the page.

With above exercise you will get understanding on how Sling model works. Follow the Sling documentation for sling models annotations.

Exercise 5: Write business logic into your sling model and access it on page

In this exercise we will write a new sling model and use that in a new component which we will create as part of this exercise. This exercise will prove your understanding of Sling models.

1. Create a new component with the name as **mylearning**. Now since you are familiar with creation of the component there are no steps provided to create this component.

2. Get two number fields into component's dialog which will take two numbers as input from the dialog.

3. In component's html you have to display the value of these two numbers which you will read via sling model.

4. Write your own sling model class "**FirstModel.java**" inside the model package. Which will get values injected from the component's dialog and pass them to sightly/html via public getter methods.

5. For your reference code is provided into the exercise folder for the component and sling model.

Sling models explanation will be provided from this class which you will be creating in this exercise.

Exercise 6: Various Type of Java classes which will be used into actual projects

Objective of this exercise is to go through the different type of java classes. We will walk through them from the sample classes.

Actual Uses of all such classes will be provided in further exercises.